

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

Yeremey Pervunetskikh

Fac. of Computer Science

HSE University

Pokrovsky Blvd, 11, 109028, Moscow, Russia

Email: eremejperv@gmail.com

Abstract—This paper adapts the Tsay-Li framework to transactional memory models. Most modern concurrent algorithms are either lock-based or complex, hard-to-understand lock-free structures, while Software Transactional Memory (STM) remains underutilized. STM facilitates the non-blocking implementation of complex algorithms through transaction mechanisms. Originally designed to grant lock-free properties to Binary Search Trees, Tsay-Li framework is used here to demonstrate how formal specifications can prove that universal algorithms maintain correctness when constructed from specialized components. The specifications are developed using the TLA+ language and the TLC model checker. The resulting specialization demonstrates that lock-based and non-blocking algorithms share more structural similarities than their source code suggests. This insight supports the creation of universal constructions for achieving lock-free properties. This work is intended for researchers seeking new universal paths to non-blocking algorithm implementations.

Index Terms—concurrent data structures, TLA+, formal verification, non-blocking algorithms, binary search trees, transactional memory.

I. INTRODUCTION

In recent decades, numerous concurrency approaches have been developed to provide high throughput with minimal system overhead. One desired example is the framework by J. Tsay and H. Li [1], along with its adaptation for red-black trees and modern architectures by A. Natarajan and N. Mittal [2]. The core principle of this framework is to bring lock-free properties to binary search trees. This guarantees that at least one thread will always complete its execution in a finite number of steps. To achieve this, Natarajan and Mittal introduced a descriptor-based approach that ensures the structural consistency of the data.

In this paper, we propose a generalization of these approaches to build a Software Transactional Memory (STM) framework. Transactional memory [3] enables non-blocking operations, which increases throughput and reduces the latency typical for traditional locking protocols. Certainly, system consistency is maintained. Furthermore, STM workflow introduces key advantages, such as support for nested transactions, which have no direct equivalent in lock-based systems. This achievement falls outside the scope of this work, but design considerations will be discussed further.

The core mechanism of a framework relies on executing transactions, which are independent operations performed si-

multaneously on data. During execution, threads create local copies of the data they modify. These changes are later committed to global memory in a way that prevents interference. If a conflict occurs, the unsuccessful transactions are aborted. Generally, this framework is divided into two primary implementations: lock-based (blocking) and non-blocking. The fundamental difference lies in the progress guarantees they provide, specifically ensuring that the system as a whole continues to make progress even if individual threads are stalled or delayed.

Universal algorithms, derived through the transformation and generalization of specialized approaches, remain highly functional and correct. The primary objective of this paper is to demonstrate how formal specifications can validate this assertion, specifically for STM and the Tsay-Li framework. To work with these mathematical descriptions, we propose utilizing the TLA+ specification language.

TLA+ (Temporal Logic of Actions) [4] is a formal specification language developed by Leslie Lamport for modeling concurrent and distributed systems. It enables a system to be described as a state machine, allowing its safety and liveness invariants to be verified through exhaustive state space exploration. Using TLA+ as a “tool for thinking” is a well-established practice among engineers due to its rigorous verifiability. Consequently, it was chosen to specify the generalized algorithm resulting from our transformation, thus confirming its correctness under high contention.

The remainder of this paper is organized as follows. Section II reviews existing concurrent frameworks and discusses the rationale behind their design choices. Section III outlines the underlying mechanism for lock-free binary search trees; a link to the primary specification file is provided in Section VII. Section IV presents the core contribution of this work, detailing the transformation of the frame-based approach into a STM algorithm. Section V analyzes the internal components, while Section VI establishes the scope of our verification results.

II. RELATED WORKS

The protocol for software transactional memory presented here was originally described in the early work of N. Shavit and D. Touitou [3] and subsequently optimized by Guerraoui et al. [5]. Our primary focus is to formally verify this protocol and demonstrate that it can be derived from lock-free

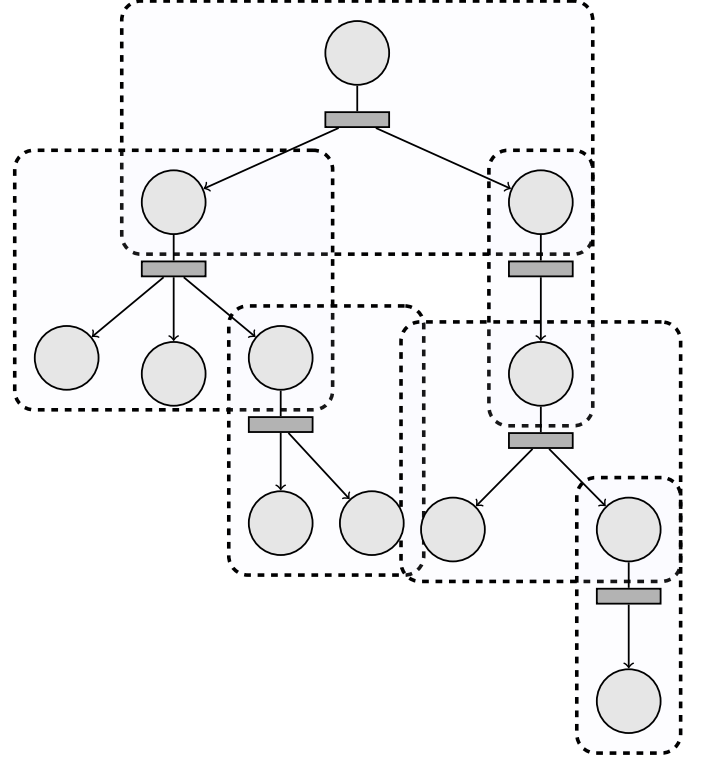
frameworks designed for complex data structures; conversely, specialized models can be instantiated from a generalized STM construction and accelerated for targeted performance. While the original specifications in the aforementioned frameworks are presented as textual descriptions with theorem-based proofs, this approach straightforward to implement can introduce verification challenges. To address this, we propose using TLA+ to provide a mathematical validation of these assertions.

Alternative STM frameworks exist, such as those by Ennals [6] and Harris [7], which rely on locking when operations conflict. In these systems, memory writes are performed by temporarily locking the target memory region, writing the values directly, and recording them in an undo log. Other strategies involve acquiring locks during the commit phase, restricting locking to that specific window. In contrast, our objective is to eliminate blocking operations entirely, allowing threads to operate independently. This independence is largely facilitated by a helping mechanism [8], a technique widely adopted across numerous non-blocking algorithms.

Furthermore, the described mechanism does not inherently support nested transactions [9]. Introducing nesting requires generalizing the concept of a “memory cell” within the global set. As detailed below, nodes can store references to operation descriptors that encapsulate their internal execution states. However, in nested transactions, recursive helping mechanisms can cause a thread to attempt to help itself on the same set of data. To bypass this issue, one can employ references to other descriptors within the local states of existing descriptors. Although this approach is highly complex to implement, the finite nesting depth ensures it remains linearizable, much like the base algorithm. This holds true because double references cannot be processed alongside standard references within the same execution phase. Nevertheless, evaluating this specific extension falls outside the scope of this paper; thus, its detailed verification is not presented in this work.

III. ADVANCED TSAY-LI FRAMEWORK

The work of Tsay and Li [1] was developed to ensure that even complex data structures like binary search trees could support lock-free properties while remaining linearizable. Simpler structures, such as linked lists, do not require such development; for instance, Heller et al. [10] achieved this by utilizing logical deletion. Non-blocking goal is to guarantee that at least one thread will infinitely often complete its data structure method execution. At the same time, we require these methods to be representable as atomic operations, preserving all partial execution orders. To achieve this, a frame-based approach was proposed. In this context, a frame is defined as a subtree section that originates at a specific node and is bounded below by a set of terminal nodes. The frame size corresponds to the number of elements it contains, encompassing all edges connecting the constituent nodes of the original tree. A frame is selected such that its complete atomic replacement preserves the internal invariants of the underlying data structure.



The framework partitions the nodes accessed by each data structure operation into such frames. For example, to insert a node into the tree, the algorithm clones a frame that originates at the local root node. This frame encompasses several additional nodes required to guide the insertion operation as it traverses down the data structure. Thereafter, the target operation is either performed on these nodes or they remain unchanged; the frame is then atomically replaced, provided that no other concurrent thread has modified this specific set of nodes and edges. If a concurrent modification is detected, the transformation process is retried from scratch. Otherwise, as the operation traverses deeper, a new frame is allocated. This new frame is guaranteed to overlap with the previous one, ensuring that when it is replaced by its local copy, the nodes maintain their connectivity to the root and remain continuously accessible.

This implementation was designed for the DEC Alpha architecture [11], which allowed verification of whether another thread had modified a memory element. This capability is equivalent to executing multiple atomic load-linked/store-conditional (LL/SC) instructions. Modern computer architectures, however, lack such primitives; instead, the compare-and-swap (CAS) atomic operation is predominantly used. CAS accepts three arguments: the memory location to be updated, the expected value, and the desired new value. It possesses an infinite consensus number [8], enabling non-blocking synchronization for an infinite number of threads. If a thread fails to update the value due to a mismatch with the expected value, CAS returns false and no replacement occurs; otherwise, the function returns true.

Based on these operations, Natarajan et al. [12] proposed

decomposing vertices and their subtrees into distinct, interconnected nodes. Under this approach, to atomically replace a frame with its local copy, a CAS operation needs to be executed only on the single vertex from which all other vertices in that subtree section are reachable. This unique vertex serves as the root of the frame. If the CAS on the root fails, it implies that another concurrent thread has modified it first, requiring the current thread to either retry the execution or raise an error.

However, a critical issues arises if a concurrent thread executes a CAS on an internal node rather than the frame root, compromising consistency. This internal update cannot detect the root-level CAS that replaces the entire frame. Consequently, the concurrent thread becomes disconnected from the active topology, as its target nodes are permanently bypassed by the new local frame copy. To prevent this isolation, method descriptors were introduced.

To execute an operation, a thread creates a descriptor encapsulating its current data structure operation. Afterwards, a reference to this descriptor is attached to all nodes involved in the operation. If another thread attempts to access a marked node, it must first inspect the descriptor and help the original thread complete its execution. This helping mechanism must be applied recursively if executing the helping operation requires assisting a third operation, and so on. This helping mechanism, akin to the multi-word CAS approach by Ellen et al. [13], ensures both system-wide progress and strict linearizability [14] during concurrent modifications.

We formalize this frame-based update in TLA+ to ensure its correctness. The `TreeCAS` module is modeled around a minimal tree consisting of a root and its child leaf. This structure is sufficient to verify the framework's correctness, as the critical race condition to evaluate involves a larger frame overlapping a smaller one. Furthermore, scaling the state space to include more nodes does not break the framework, as confirmed by our larger-scale model checking runs. The only part of the spec needs to replace is `InitialTree` internal variable in `Init` statement (Except for `Nodes` in config file).

The model state is mainly defined by variables: `nodeStruct` and `structLinks` map nodes to their internal structures and target child nodes; `nodeDesc` tracks descriptor references (or `NULL`) within cells; `descState` manages the descriptor lifecycle (`IDLE`, `ACTIVE`, `COMMITTING`, `SUCCESS`, `ABORTED`); and `descPending` along with `descFrame` track unacquired nodes and assigned frame boundaries for each descriptor.

To model the allocation of new nodes, we utilize an infinite memory pool constrained by a state-space filter limiting the maximum number of active allocated cells to 6. The overall algorithm comprises 5 distinct phases, with the 3rd phase implementing the recursive helping mechanism. This phase is structured as follows:

```
Cooperatively advance for descriptor if it blocks our path
Help(d, n)  $\triangleq$  LET otherD  $\triangleq$  nodeDesc[n] IN
   $\wedge$  descState[d] = "ACTIVE" We can only offer help if
   $\wedge$  n  $\in$  descPending[d] we are actively trying to acquire nodes
```

```
 $\wedge$  nodeDesc[n]  $\neq$  NULL The target node must be currently
 $\wedge$  nodeDesc[n]  $\neq$  d locked by a different descriptor otherD
 $\wedge$  IF descState[otherD]  $\in$  {"ACTIVE", "COMMITTING"}
THEN If the competitor is in progress
  CommitStep(otherD) then make the next step
ELSE IF descState[otherD]  $\in$  {"SUCCESS", "ABORTED"}
THEN Help it unlock nodes and free memory
  ReleaseDescriptor(otherD)
ELSE If otherD is IDLE, do nothing
UNCHANGED vars
```

As demonstrated here, we do not distinguish between the concept of an execution thread and the descriptor of the operation it performs. This abstraction can be interpreted as the use of coroutines within the physical production code, a pattern that is prevalent in practice. An interesting aspect of constructing formal specifications for non-blocking algorithms is that the `Help(d, n)` routine is only strictly necessary in the actual implementation. While omitting it compromises the global progress property, it is not mathematically required to maintain safety invariants.

GlobalProgress $\triangleq$

```
 $\exists d \in Descs : descState[d] = \text{"ACTIVE"} \leadsto$ 
 $\exists u \in Descs : descState[u] = \text{"SUCCESS"}$ 
 $\vee \neg FreeStructConstraint$ 
```

To verify linearizability, we enforced the preservation of the tree topology invariant, ensuring that every node remains continuously reachable from the root and is never erroneously obscured by a concurrent transaction.

Another aspect of the specification is the CAS acquisition phase. If the linked structure of a target node has been modified, our transaction must be aborted. The core block for this mechanism is presented below.

```
 $\wedge$  IF nodeStruct[n]  $\neq$  descExpectedStruct[d][n]
THEN The underlying node structure has changed since
  the descriptor creation. Abort the transaction.
   $\wedge$  descState' = [descState EXCEPT ![d] = "ABORTED"]
   $\wedge$  UNCHANGED vars \ { ... }
ELSE IF nodeDesc[n] = NULL
THEN ExecuteAcquire(d, n) CAS SUCCESS: Lock the node
ELSE UNCHANGED vars CAS FAIL: Node is occupied
```

Next, we can examine the step by step committing phase. It preserves the structural consistency invariant because accessing any modified node requires interacting with the operation descriptor, enforcing participation in the next commit step and maintaining consistency.

IV. ADAPTATION TO TRANSACTIONAL MEMORY

A. Introducing Generalisation

In previous model, locating a target node requires a top-down traversal where a thread must observe all active transactions within its path. Formally, ensuring that an update to a set is visible to all overlapping operations is non-trivial; while visibility is inherent for supersets of this set, it cannot be guaranteed for arbitrary subsets without a common synchronization point. Since no single element can consistently represent all minimal frame nodes (e.g., disjoint leaves), a global consensus

on the frame’s state is required. This formal constraint justifies reliance on localized frames and edge-based indirection as the primary mechanism for maintaining a consistent view across concurrent operations.

Therefore, we bypass the “lowest common ancestor” (LCA) approach for frame identification, as it fails to guarantee atomicity during complex rebalancing to the abstraction of frame sets. Furthermore, if the LCA is applicable, this reasoning holds; moreover, it can be optimized for specific cases, establishing a standard paradigm for implementing high-performance concurrent operations. Instead, we encapsulate within the descriptor all memory cell data accessed by the current operation. This ensures that all modifications to nodes and edges are performed strictly inside this descriptor. The primary data structure positions will store exclusively references, as their values no longer remain independently valid and are not updated atomically while an operation is in progress over them.

This two-phase approach—logical commit followed by physical propagation—mirrors the Multi-word Compare-and-Swap (MCAS) semantics proposed by Harris et al. [15], ensuring that updates to multiple memory locations remain linearizable through a single shared descriptor. By its very nature, this mechanism approximate toward the STM model, demonstrating that lock-free tree primitives can be converted into a universal construction with overhead limited to descriptor management. Furthermore, this design inherently preserves disjoint-access parallelism, ensuring that operations targeting non-overlapping memory regions execute independently. This property yields significant liveness advantages over traditional lock-based STM implementations.

Based on this descriptor-centric architecture, we decompose each transaction into three atomic phases that ensure safe state transitions across the frame:

- **Acquire.** The thread attempts to install its descriptor D into each target memory cell $R_j \in \{r_1, \dots, r_n\}$ via $\text{CAS}(R_j, \text{old}_j, D)$. If a cell is already held by another descriptor D' , the thread must first invoke a recursive helping routine to assist the conflicting operation. This ensures that acquisition is never blocked by stalled processes. Furthermore, descriptors must be installed in an ascending order based on memory addresses to prevent deadlocks.
- **Decision.** The logical linearization point occurs here. Once all target cells are successfully acquired, the thread attempts an atomic CAS on the descriptor’s status field, transitioning it from *Undecided* to either *Committed* or *Aborted*.
- **Release.** In the final cleanup phase, the thread resets each node’s state to a concrete value. If the status is *Committed*, the cell is updated to new_j ; otherwise, it is reverted to old_j . This phase effectively releases the descriptor-based ownership and makes the linearization point change visible to the rest of the system.

This approach follows the lineage of descriptor-based atomicity and was later refined in composable memory models

[9]. However, our model fundamentally alters the semantics of read operations. To guarantee a lock-free view, any process accessing a cell must proactively resolve any pending descriptor. This dependency effectively elevates a simple load into a “heavyweight” operation, as a read may now trigger a physical write to memory. While preventing transient state visibility, this imposes non-trivial overhead: every read triggers a potential global descriptor state transition, degrading performance in read-heavy workloads.

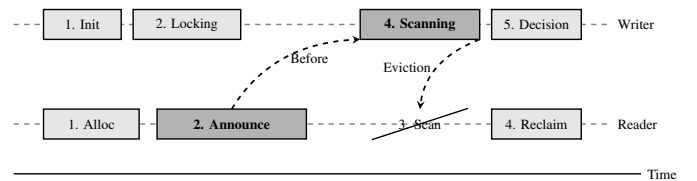
B. Strategy for a Reading Model

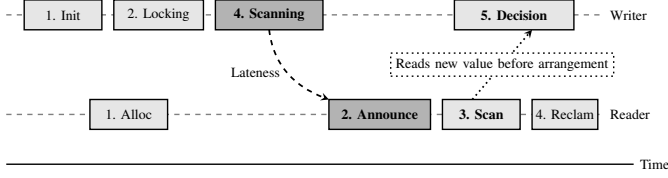
Shifting to a “lightweight” reading model requires better strategy for tracking concurrent dependencies. This concept mirrors the Linux kernel’s Read-Copy Update (RCU) mechanism [16]. We adhere to an *announce array*, where each reader thread registers its presence before executing an operation. Every writing operation must inspect this array during the atomic replacement of its cell list, subsequently aborting any reader threads that are operating on nodes within that list. This validation step must occur strictly before the atomic replacement is finalized. Otherwise, a reader thread might complete its execution before the check ends, failing to notice that its operation should have been aborted due to concurrent updates.

Since a write operation involving in aborting concurrent readers cannot be aborted itself, the new list of cell values intended for the atomic replacement is guaranteed to be globally valid for the entire data structure. Also, an incoming read operation must first register itself in the announce array before determining the current value of each target node. This value is either stored directly in the cell or contained within the descriptor referenced by that cell. The reader thread must retrieve this value or, if a write operation has already staged a new value, fetch the pending new value that is ready for installation. After completing its execution validly, the reader thread simply unregisters itself from the announce array to release its claim.

The safety of this optimization relies on a fence operation that enforces a strict happens-before relationship [17] between the reader’s *Announce* step and the writer’s *Scan* phase. This ordering preserves a critical safety invariant:

- **Writer-side visibility:** If *Announce* precedes *Scan*, the writer is guaranteed to observe the reader’s entry in the announce array. Consequently, the writing thread can safely abort the conflicting reading operation.
- **Reader-side awareness:** Conversely, if *Scan* completes before *Announce*, the reader is guaranteed to observe the writer’s newly installed value (or an even newer version) during its read phase.





Memory barriers [18] prevent reorderings from introducing stale reads. Consequently, the algorithm maintains linearizability while eliminating forced global writes for reads, significantly reducing contention.

V. FORMAL METHODOLOGY

To verify the MCAS algorithm, we developed a formal specification in TLA+ suitable for the TLC model checker. Core memory is tracked via `nodeValue` and `nodeDesc`, which map nodes to their current values and assigned write descriptors. First and further, `freeWDescs`, `freeRDescs`, and `successCount` manage resource allocation and metrics, while `absNodeValue` provides a refinement mapping to verify linearizability against an ideal atomic memory.

To maintain a finite state space during model checking, we enforce an execution boundary via the constraint `countConstraint == successCount <= 3`. Furthermore, since the foundational algorithmic primitives remain structurally unchanged, we employ an identical to `TreeCAS` module global progress property to verify liveness (and consider on lock-freedom).

A. Writer Lifecycle: Acquisition and Atomic Commit

Write transaction states are managed by `wDescState`, `wDescPending`, `wDescFrame`, `wDescOldValue`, and `wDescNewValue`, capturing descriptor lifecycles, unacquired cells, locked frames, and value snapshots. Parallel writer actions are structured into seven distinct phases.

The execution begins with initialization, which allocates a unique write descriptor for the active transaction, followed by the acquiring phase to sequentially link and acquire target frame cells. Next, the reader eviction phase broadcasts the `SCANNING` state to reject ongoing stalled readers before values shift. The transaction then enters the decision phase to reach the logical commit boundary, making target values ready for physical memory persistence. Finally, the reclamation phase performs a lazy cleanup and releases the descriptor. Subsequently, a cooperative help operation can execute at any phase to assist conflicting descriptors; this step is optional for basic specification safety but strictly required to ensure lock-freedom.

It is worth noting that the difference between these two specifications lies in the extensions introduced in the preceding sections. Therefore, the core components remain valid, with the only modification being the elimination of node-structure indirection.

B. Reader Lifecycle: Announcement and Validation

Read operations utilize an announce array represented by `rDescState`, `rDescFrame`, and `rDescView` to track

reader status, target frames, and local snapshots. Reader actions are structured into four phases.

The process begins with allocation, which initializes the read descriptor with a target frame, followed by the announcement phase to publish the operation to the announce array (which implemented as `ANNOUNCED` state). This step allows writers to reject conflicting reads. Next, the scan phase reads memory cells.

C. Formal Verification and Correctness

The correctness of the state-transition system is rigorously checked using two primary safety invariants that enforce linearizability across concurrent phases:

- **AtomicityInvariant:** This invariant ensures the global memory consistency between the physical and abstract states. If a memory cell is not held by any descriptor (`NULL`), its physical value (`nodeValue`) must perfectly align with the ideal atomic representation (`absNodeValue`). When a cell is owned by an active descriptor, its value must match the abstract state if the transaction is `ACTIVE` or `ABORTED`. If the transaction progresses to the `SCANNING` or `SUCCESS` phases—where the abstract state has already linearized—the physical memory is allowed to retaining its original `wDescOldValue` or completely match the newly updated abstract state.
- **ReaderAtomicity:** This invariant validates snapshot consistency for reading operations. Evaluated exclusively after a reader transitions to the `READING` state, it guarantees that for every node within its target frame, the collected local view (`rDescView`) remains linearizable. Specifically, the observed value must either correspond to the current globally valid abstract memory state or match a historical snapshot (`wDescOldValue`) from a concurrently committed successful transaction that intersected with the reader's frame.

An crucial observation is that descriptors are assigned exclusively in ascending order to prevent deadlocks, mirroring the strategy used in the `TreeCAS` module:

2) **AcquireNode:** CAS-based cell ordering with descriptors

$\wedge n = (\text{CHOOSE } \text{minNode} \in wDescPending[d] :$

$\forall \text{anyNode} \in wDescPending[d] : \text{minNode} \leq \text{anyNode})$

Furthermore, the presented algorithm is theoretically designed to never produce aborted transactions under the verification configuration. Although it maintains full support for aborts if required by the top-level algorithm using the framework. The alternative implementation below violates the lock-freedom property:

```

MODULE AcquireNode(d, n)
  \w IF nodeValue[n] \neq wDescOldValue[d][n]
  THEN  CAS failed: abort the transaction and
  \w wDescState' = [wDescState EXCEPT ![d] = "ABORTED"]
  restore the pending frame
  \w wDescPending' = [wDescPending EXCEPT ![d] = wDescFrame[d]]
  \w UNCHANGED ...

```

The `WriterScanAndReject` function (Phase 4) encapsulates both the abortion of conflicting read operations and the logical linearization point. It is implemented as follows:

```

MODULE WriterScanAndReject(d)
LET conflictingReaders  $\triangleq$  {r  $\in$  RDescriptors :
  rDescState[r]  $\in$  {"ANNOUNCED", "READING"}  $\wedge$ 
  (rDescFrame[r]  $\cap$  wDescFrame[d])  $\neq$  {}} IN
 $\wedge$  absNodeValue' = [n  $\in$  Nodes  $\mapsto$ 
  IF n  $\in$  wDescFrame[d]
  THEN wDescNewValue[d][n]
  ELSE absNodeValue[n]]
 $\wedge$  IF conflictingReaders  $\neq$  {}
  THEN rDescState' = [r  $\in$  RDescriptors  $\mapsto$ 
    IF r  $\in$  conflictingReaders
    THEN "REJECTED"
    ELSE rDescState[r]]
  ELSE UNCHANGED rDescState

```

The complete specifications for both modules are available on GitHub, with the corresponding repository link provided in Section VII.

VI. EVALUATION

The evaluation demonstrates that after adapting the frame-replacement operations into generalized transactions, the framework remains correct, satisfying both linearizability and lock-freedom, with the latter expressed via the `GlobalProgress` property. Verification was conducted using exhaustive state-space exploration under our pre-defined execution constraints.

Since the execution of all data structure methods is ultimately decomposed into phases that can overlap, verification using two concurrent threads is sufficient. This setup confirms that any execution path under arbitrary instruction interleaving remains correct. Introducing a third thread merely adds a phase that overlaps with the other two in the exact same manner as they do with each other.

A larger number of threads exponentially expands the state space, making exhaustive exploration by the TLC model checker computationally prohibitive; thus, we opted to limit the configuration to two writing threads and single reading one. Similarly, to verify whether the framework can handle overlapping frame sections correctly, a memory pool of two cells is fully sufficient. The same reasoning applies to the tree module.

VII. CONCLUSION

This work adapts the Tsai-Li framework for tree-based data structures to modern universal Software Transactional Memory Model. Our TLA+ analysis confirms that the proposed lightweight reading protocol—utilizing an Announce-Array and barriers maintains strict linearizability while significantly reducing helping overhead. This model demonstrates a successful balance between formal robustness and modern hardware constraints.

The source code for this project is available at:

<https://github.com/Yerk08/MCAS-formal-specification/>

REFERENCES

- [1] J. J. Tsay and H. C. Li, "Lock-free concurrent tree structures for multiprocessor systems (extended abstract)," in *Proceedings of the 1994 International Conference on Parallel and Distributed Systems*, 1994, pp. 544–549. doi: 10.1109/ICPADS.1994.590370.
- [2] A. Natarajan and N. Mittal, "Fast concurrent lock-free binary search trees," *ACM SIGPLAN Notices*, vol. 49, no. 8, pp. 317–328, Feb. 2014. doi: 10.1145/2692916.2555256.
- [3] N. Shavit and D. Touitou, "Software transactional memory," in *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing (PODC '95)*. New York, NY, USA: ACM, 1995, pp. 204–213. doi: 10.1145/224964.224987.
- [4] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Boston, MA, USA: Addison-Wesley, 2002.
- [5] R. Guerraoui, A. Kogan, V. J. Marathe, and I. Zabolotchi, "Efficient multi-word compare and swap," in *Proceedings of the 34th International Symposium on Distributed Computing (DISC 2020)*. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 4:1–4:18. doi: 10.4230/LIPIcs.DISC.2020.4.
- [6] R. Ennals, "Software transactional memory should not be lock free," Intel Research Cambridge, Tech. Rep. IRC-TR-06-052, 2006.
- [7] T. Harris and K. Fraser, "Language support for lightweight transactions," in *Proceedings of the 18th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '03)*, 2003, pp. 322–335. doi: 10.1145/949305.949340.
- [8] M. Herlihy, "Wait-free synchronization," *ACM Transactions on Programming Languages and Systems*, vol. 13, no. 1, pp. 124–149, Jan. 1991. doi: 10.1145/114005.102808.
- [9] T. Harris, S. Marlow, S. Peyton-Jones, and M. Herlihy, "Composable memory transactions," in *Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '05)*. New York, NY, USA: ACM, 2005, pp. 48–60. doi: 10.1145/1065944.1065952.
- [10] S. Heller, M. Herlihy, V. Luchangco, M. Moir, W. N. Scherer III, and N. Shavit, "A lazy concurrent list-based set algorithm," in *Principles of Distributed Systems (OPDIS 2005)*, M. K. Aguilera, Ed. Berlin, Heidelberg: Springer, 2005, pp. 3–16. doi: 10.1007/11795490_3.
- [11] R. L. Sites and E. McLellan, "The Alpha AXP Architecture and 21064 Processor," *IEEE Micro*, vol. 12, no. 6, pp. 10–11, Dec. 1992. doi: 10.1109/40.216747.
- [12] A. Natarajan, L. H. Savoie, and N. Mittal, "Concurrent wait-free red black trees," in *Stabilization, Safety, and Security of Distributed Systems (SSS 2013)*, T. Higashino, Y. Katayama, T. Masuzawa, M. Potop-Butucaru, and M. Yamashita, Eds. Cham, Switzerland: Springer, 2013, pp. 45–60. doi: 10.1007/978-3-319-03089-0_4.
- [13] F. Ellen, P. Fatourou, E. Ruppert, and F. van Breugel, "Non-blocking binary search trees," in *Proceedings of the 29th ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing (PODC '10)*. New York, NY, USA: ACM, 2010, pp. 131–140. doi: 10.1145/1835698.1835736.
- [14] M. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, Jul. 1990. doi: 10.1145/78969.78972.
- [15] T. L. Harris, K. Fraser, and I. A. Pratt, "A practical multi-word compare-and-swap operation," in *Distributed Computing (DISC 2002)*, D. Malkhi, Ed. Berlin, Heidelberg: Springer, 2002, pp. 265–279. doi: 10.1007/3-540-36108-1_18.
- [16] P. E. McKenney and J. Slingwine, "Read-Copy Update: Using Execution History to Solve Concurrency Problems," in *Proceedings of the 11th International Parallel and Distributed Processing Symposium (IPDPS)*, 1998, pp. 509–518.
- [17] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978. doi: 10.1145/359545.359563.
- [18] P. E. McKenney, "Memory barriers: a hardware view for software hackers," Linux Technology Center, IBM Beaverton, Tech. Rep., 2010.